



Amrita School of AI
COIMBATORE CAMPUS

UNIT 3 · SESSIONS 20-25

GPUs and CUDA

A processor that is bad at everything except the one thing you need.

Dr. Abhijith Anandakrishnan · Assistant Professor
23AID304 · Odd Semester 2026-27

CPU vs GPU

- **CPU core**

Spends silicon on making **one** stream fast: branch prediction, out-of-order, big caches.

Goal: **minimum latency**

- **GPU SM**

Spends silicon on arithmetic units and registers. Hides stalls by having thousands of threads ready.

Goal: **maximum throughput**

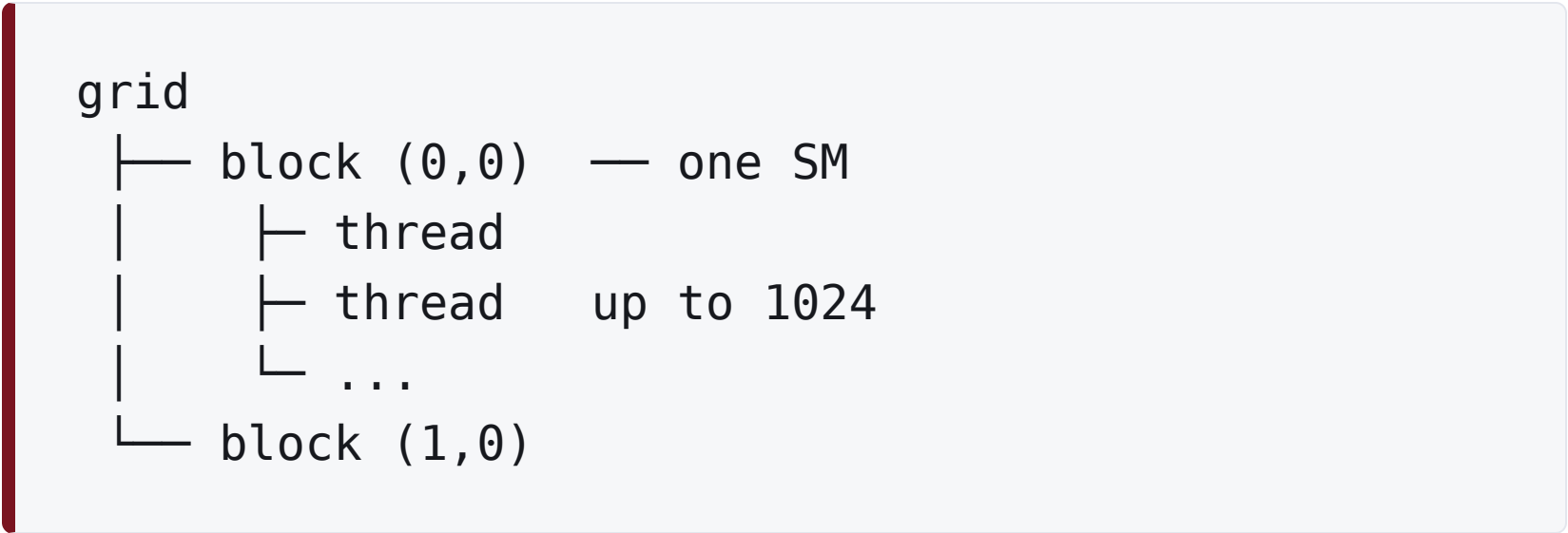
EPYC 9654 vs A100 80GB

	CPU	GPU
threads in flight	~192	~220 000
FP64 peak	~4 TF/s	9.7 TF/s
bandwidth	~400 GB/s	~2000 GB/s

The bandwidth row is why it is worth moving your data across PCIe at all.

Five times the memory throughput of a large server socket.

Thread, warp, block, grid



LEVEL	CO-OPERATE?
thread	with its block
warp (32)	in lockstep
block	shared mem + <code>__syncthreads</code>
grid	no

Blocks must be independent.

That is what lets the same binary scale from 20 SMs to 108.

32 threads, one instruction pointer

```
if (threadIdx.x % 2)  
    heavy_a();  
else  
    heavy_b();
```

The warp runs **both** branches and masks the inactive lanes.

Cost is the **sum**, not the max.

WARP DIVERGENCE

Which branches are free?

- **Free**

`blockIdx.x % 2` — whole blocks agree

`threadIdx.x / 32` — whole warps agree

- **Costs double**

`threadIdx.x % 2` — splits every warp down the middle

The idiom, and the guard

```
__global__ void vec_add(const float *a, const float *b, float *c, int n)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) c[i] = a[i] + b[i];    // the guard is NOT optional
}

int blocks = (n + threads - 1) / threads;    // ceiling division
vec_add<<blocks, threads>>>(da, db, dc, n);
```


All three are silent

- **No bounds guard**

Ceiling division makes extra threads. They write past the array. Memory corrupted, no error.

- **Assuming it finished**

A launch is **asynchronous**. Timing without `cudaDeviceSynchronize()` measures the launch.

- **Not checking errors**

Launch failures surface later, at an unrelated call. Wrap everything in `CUDA_CHECK`.

The hierarchy again, sharper

SPACE	LATENCY	SIZE
registers	~1 cy	255/thread
shared	~20 cy	164 KB/SM
L2	~200 cy	40 MB
global	~400 cy	80 GB
host over PCIe	~μs	system RAM

PCIe: ****25 GB/s**** A100 memory: ****2000 GB/s****
Eighty times. Copy in, one cheap kernel, copy out is ****slower than the CPU****.

Data must stay resident.

A GPU pays off when the transfer is amortised over many kernels.

Coalescing

```
// COALESCED: one 128-byte transaction  
out[i] = in[i];
```

```
// NOT: up to 32 transactions  
out[i * stride] = in[i * stride];
```

32 threads reading 32 consecutive words → **one** transaction.

32 scattered words → up to **32**.

THIS IS UNIT 0'S CACHE LINE, AGAIN

When one access must be strided, route through shared memory

```
__shared__ float tile[32][33];  
  
tile[ty][tx] = in[y*n + x];    // coalesced  
__syncthreads();  
out[y*n + x] = tile[tx][ty];    // coalesced
```

Both global accesses now coalesce; the awkward transposition happens on chip.

Typically **5-10x**.

The `33` is not a typo →

ONE CHARACTER, OFTEN 2X

Shared memory bank conflicts

32 banks, 4 bytes wide, interleaved.

`tile[32][32]`: a whole column lands in **one bank**. 32-way serialisation.

`tile[32][33]`: each row shifts by one, so a column spreads across all 32 banks.

PADDING

Tiled matrix multiply

Each block loads a $\text{TILE} \times \text{TILE}$ tile of A and B into shared memory.

Each element is then used **TILE times** instead of once.

I rises from **0.25** to about **$\text{TILE}/4$** .

With $\text{TILE}=32$ that is 32x, which moves the kernel from far left of the ridge to the right of it.

ROOFLINE, AGAIN

BOTH ARE NECESSARY

The two `\_\_syncthreads()`

First: everyone finished *writing* the tile before anyone *reads* it.

Second: everyone finished *reading* before the next iteration *overwrites* it.

Remove either → intermittently wrong answers, more often on larger matrices. And it must be reached by **every** thread. Inside a divergent `if`, it hangs the block.

MEASURED ON OUR OWN RTX 6000 ADA, 4096², FP32

The textbook number is wrong. Here is the real one.

VERSION	MS	GFLOP/S	VS NAIVE
naive	24.4	5 628	1.00x
tiled	19.4	7 071	1.26x
cuBLAS	2.2	63 302	11.3x

Books say tiling gives 10x. We measured ****1.26x****. Huge L2 absorbs the re-reads, and the naive kernel was already coalesced.

cuBLAS is **9x faster than your best tiled kernel**. Write the library call first.

More resident warps is a means, not an end

- **What it is**

Resident warps $\div$ maximum. It exists to **hide memory latency**.

Limited by registers, shared memory, block size.

- **The myth**

"Higher occupancy is always better."

A register-heavy kernel at 25% occupancy can beat a 100% one that reloads from memory.

Read the tools in this order

1. Nsight Systems — is the GPU even busy?

Gaps in the timeline mean the problem is host-side. No kernel work will help.

2. Nsight Compute — once one kernel dominates.

It draws the roofline for you, measured.

What the counters mean

SYMPTOM	MEANS
DRAM ~peak, SM low	memory bound, at the limit
both low	latency bound
SM ~peak	done

SYMPTOM	ACTION
memory bound	tile, fuse, lower precision
latency bound	more occupancy, more work per thread
uncoalesced	restructure indexing

Next: many GPUs, and someone else's datacentre.